

In-Memory Account Ledger Core

Architecture and Trade-offs

Dmitry Bannikov

10 September 2026

1. Append-only at scale

What breaks first at 100× volume

Not memory. A six-day window of entries is small even at a hundred times the volume, and the log is the cheapest thing in the system to store. What breaks is the *daily close*, and it breaks quadratically.

Every balance in this design is derived by scanning the log for entries with *value_date* $\leq d$. That is $O(N)$ in the number of entries. The close calls it once per account per candidate day, and under the retroactive fee policy the candidate set is every day back to the start of the window, so the close is $O(D \cdot A \cdot N)$. Since N itself grows with volume, multiplying throughput by 100 multiplies close-time work by roughly 10^4 . A close that takes seconds today takes hours, and it takes them inside the overnight window where there is no slack.

The second thing to break is worse, because it is on a latency budget rather than a batch window: the available-balance check on the authorisation path. Deriving a hold's current state means scanning the hold log for that authorisation, and summing active holds means doing that once per authorisation the account has ever had. The cost of approving a card transaction therefore grows with the account's *lifetime* authorisation count, not with the number of holds actually outstanding. An account that has been open a year is slower to authorise than one opened yesterday holding the same funds. That is the wrong quantity to be sensitive to, and it degrades silently until it starts timing out at the scheme.

Where unbounded state accumulates

The log grows without bound, and that is correct: it is the record, and I would not bound it. The defect is not unbounded storage, it is *unbounded read amplification* — three places where the cost of answering a bounded question grows with total history:

- Every balance query reads from the beginning of time. There is no lower bound on the scan, so answering “what is the balance today” gets more expensive every day, forever.
- Settled, released and declined authorisations never leave the hot path. They are dead records that every subsequent approval decision still pays to skip.
- The close-time snapshot map grows one entry per day per account, and nothing ever reads the old ones except a report.

The cheapest change that defers it

In two tiers, because the cheap fix and the right fix are not the same fix.

Immediately, and for almost nothing: maintain a per-(account, value date) delta bucket on append and derive balances by prefix-summing at most D buckets rather than N entries. A backdated entry invalidates only the prefix sums at or after its value date. This turns the close from quadratic in volume into linear in days and touches no semantics, because the index is still *derived* — it can be rebuilt from the log, and a test that asserts the index equals a full replay is a property the design already needs for other reasons. Partitioning holds into open and closed sets is the same kind of change and removes the authorisation-path problem entirely.

Structurally: materialise a cumulative balance checkpoint per account at period boundaries and read *checkpoint plus delta*. The obstacle is that a checkpoint is only sound if the period behind it cannot change, and value-dating is precisely the mechanism by which it can. So the checkpoint is not really an indexing change — it is a policy change wearing an indexing costume, and the policy is a *correction window*: past some cut-off, an entry value-dated into a closed period is refused at intake and rebooked as an adjustment in the open period, carrying the original date as data rather than as position.

That is the change I would argue for first, because it is the only one that pays twice. It bounds the scan, it makes the ledger reconcilable against a general ledger whose periods are already hard-closed, and it bounds the retroactive recomputation that my own failing test exposes — where the fees a customer pays depend on the order corrections arrived in rather than on the contents of the log. One control, two classes of problem.

2. Value-dated entries in production

Operational surface

A value-dated entry is a licence to rewrite a day that has already been reported, reconciled, and in most cases sent to the customer. Everything downstream of a closed day becomes mutable with it.

- **Statement restatement.** A statement is a customer-facing document with legal weight. A backdated entry makes an issued statement wrong, which means the bank needs statement versioning, correction advices, and the ability to reproduce a statement exactly as it was issued — not merely as the ledger reads now. This is why the report in Part 1 keeps both the close-time figure and the restated one; the pair is the minimum needed to explain a charge to the person who received it.
- **Prior-period adjustment.** The general ledger closes on a calendar; the core ledger, as built, does not. A backdated entry that crosses a month end lands in a GL period that no longer accepts postings, and one that crosses a year end touches audited financials. Without an explicit adjustment path the two ledgers simply diverge, and the difference surfaces as an unexplained suspense balance.
- **Retroactive charges.** The highest-risk output of the whole mechanism. The CBUAE Consumer Protection Regulation (Circular 8/2020) and its accompanying Standards require charges to be disclosed, transparent and explainable, with a complaints route. A fee levied for a day whose issued statement showed the account in credit cannot be explained from any document the customer holds.
- **Tax periods.** Explicit fee income carries 5% VAT. A fee dated into an already-filed period changes a filed return.
- **AML and monitoring.** Transaction monitoring aggregates by date; a backdated entry can retrospectively change a threshold aggregation that a structuring rule already evaluated. Under Federal Decree-Law 20/2018 the record must also survive five-year retention in both its original and corrected form — the correction does not replace the original, and the system has to be able to produce both.
- **Internal fraud and earnings management.** Whoever chooses a value date chooses the fee, the interest and the reporting period into which revenue falls. Backdating is a control surface before it is a feature, and in the model as built no event records who chose.

One control before go-live

An enforced value-date correction window with four-eyes approval and mandatory reason coding, applied at the ledger boundary.

An entry whose value date is older than a published cut-off is rejected at intake. Booking one anyway requires a second authorised approver, a reason code drawn from a closed list, and it automatically raises a customer notification whenever the entry changes an issued statement or levies a charge.

I would choose this over the obvious alternatives — a nightly restatement reconciliation report, an alerting threshold on backdated volume, a shadow GL — for three reasons. It is preventive rather than detective, so the exposure never opens rather than being discovered afterwards. It puts a named human against every retroactive charge, which is the first thing both a consumer-protection complaint and a supervisory examination ask for. And it is the same mechanism that makes checkpointing sound in §1, so it is one control answering a regulatory question and a scaling question at once.

3. Authorisation lifecycle

A matching settlement is the least common way an authorisation ends. Every other ending below is reachable in this domain, and each needs a mandated behaviour rather than a default.

One rule governs all of them, and the model as built already follows it: an ending is always an *event*, never a background sweep that mutates state. If available balance rises and no event says why, nobody can answer the customer's question. A second rule follows from the first: what is released is always the *remaining* hold, computed at that moment — never the original amount added back.

Declined at authorisation. Insufficient available balance, a risk decision, or an invalid card state. *Behaviour:* record the decline with its reason and the available balance at that instant, and keep it queryable. Declines are a top complaint category and a fraud signal; a decline that leaves no trace is an unanswerable support ticket. No hold is created.

Expiry without presentment. The acquirer never presents, or presents after the hold window — an abandoned checkout, or a pre-authorisation the merchant simply never uses. *Behaviour:* release automatically at a scheme- and merchant-category-specific window, as an explicit expiry event. A settlement arriving afterwards is not an error; it becomes a force post and is handled as one.

Reversal by the acquirer, full or partial. The terminal cancels, or an estimated authorisation is reduced to the amount actually transacted — the fuel pump that authorises an estimate and reverses the unused portion.

Behaviour: release exactly the reversed amount and leave the authorisation live at the reduced figure. Treating a partial reversal as a full one over-releases funds; treating it as invalid strands them.

Partial settlement, residual never presented. A split shipment where the remainder is cancelled. *Behaviour:* each settlement posts and decrements the hold; the authorisation ends when the hold reaches zero or the expiry window closes, whichever comes first — not on the first settlement.

Account frozen, closed, or the customer deceased. A sanctions match, a court order, or an estate. *Behaviour:* the hold must not simply be released, because releasing it makes the funds available for withdrawal. Move the held amount to a blocked position and let the eventual settlement or expiry resolve against that, not against the customer's available balance.

Stand-in or offline authorisation the issuer never saw. The scheme approved on the issuer's behalf during an outage, or the terminal authorised below its floor limit. *Behaviour:* there is no authorisation to end — the ledger learns of the obligation only at clearing, and must accept it as a force post rather than reject a payment the scheme has already guaranteed. Matching against the clearing reference, with an amount tolerance, is what separates this from a genuinely bad message.

4. What I cut, and what each cut defers

No intake idempotency. The same event replayed posts twice. Every realistic upstream — a scheme feed, a queue, a retrying client — delivers at least once, so this is a live double-charge defect rather than a theoretical one. It is the first thing I would fix, and it is cheap: a unique constraint on the event identifier, enforced by the store rather than by a read-then-write check.

No concurrency control. Single-threaded, with a sequence counter that is not atomic. Two writers interleaving would make log order stop matching audit order, which quietly destroys the one property the whole design rests on. Production needs a single writer per account or an explicit ordering service — not a lock added later.

Fees derived rather than event-sourced. Fee assessment recomputes from current balances instead of being an event carrying the as-of watermark it was computed against. This is the flaw the failing test documents: identical entries yield different fees depending on arrival order, so any rebuild or replica that re-derives fees disagrees with production. Deferred risk is a reconciliation break that appears only in disaster recovery, which is the worst time to find it.

No actor identity on any event. Nothing records who booked an entry or chose its value date. This defers exactly the control §2 depends on: four-eyes approval is unimplementable on a model that has no notion of eyes.

One settlement per authorisation; no expiry; no partial reversal. Three of the six endings in §3 are unrepresentable. Deferred risk is customer funds held indefinitely against authorisations that will never settle, and split-shipment settlements rejected as duplicates.

No fee-reversal policy. When the entry that caused a fee is reversed, the fee stands, because no policy was given and inventing one silently seemed worse than leaving it visible. Deferred risk is a consumer-protection exposure attached to an undocumented decision — the kind that is discovered by a complaint rather than by a review.

Single-currency fee schedule, no FX. A fee owed on an account whose currency the schedule does not name raises rather than guesses at a rate. Deferred risk is a hard failure on a path that a second currency pair would reach immediately; the fix is a per-currency schedule, not a conversion.

$O(N)$ balance derivation and no checkpointing. Deliberate, to keep the log the single source of truth while the design was still moving. Deferred risk is §1 in full.